



COMSATS UNIVERSITY ISLAMABAD
(Islamabad Campus)

Project Title

Phishing URL Detector

Course

Artificial Intelligence

The Project By

SP24-BCT-020 —————> Mohib Ilyass

SP24-BCT-009 —————> Dail Ahmad

Supervisor —————> Mr. Yasir Munir

Date —————> Dec 17, 2025

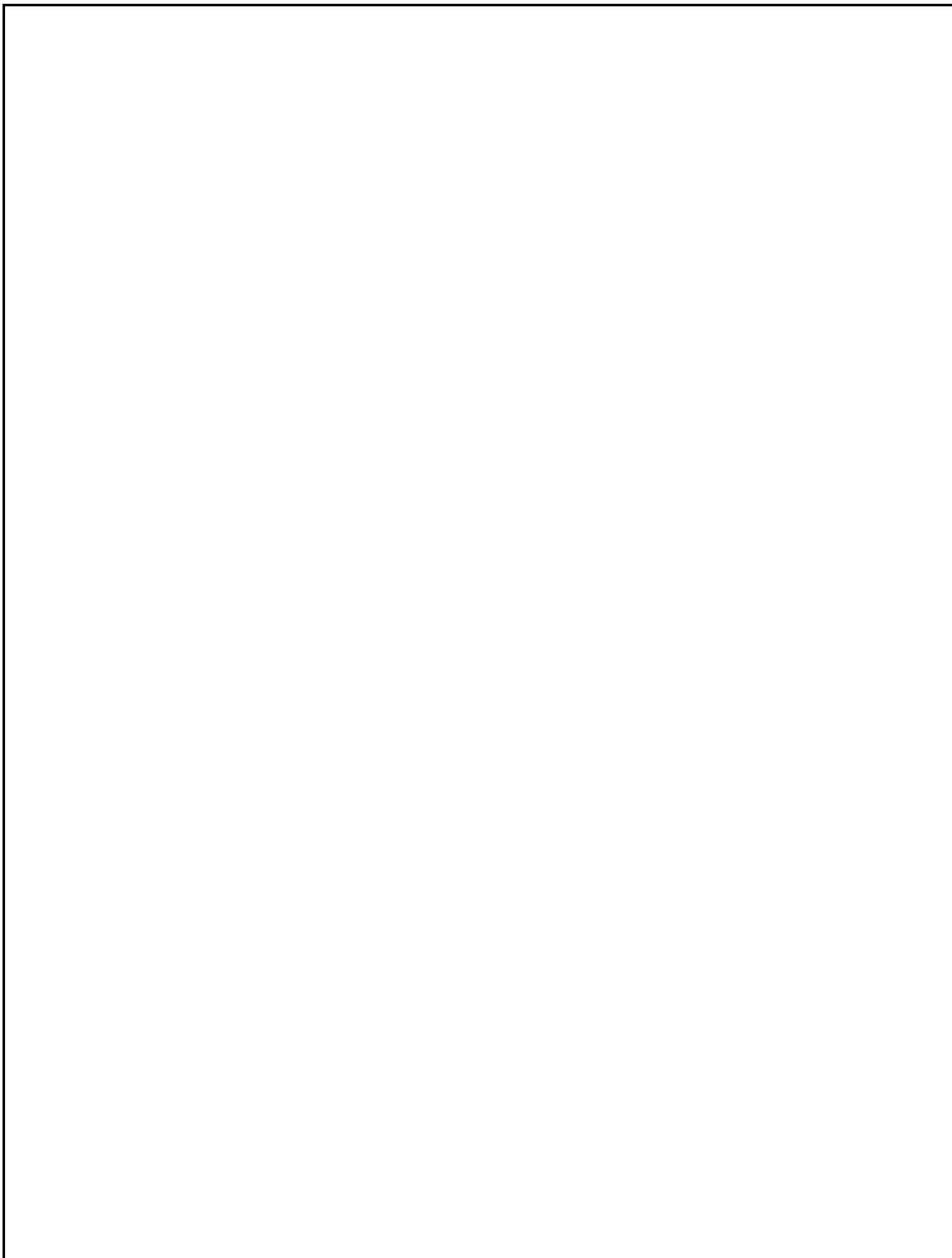


Table of Contents

PHISHING URL DETECTOR.....	4
1. ABSTRACT	4
2. INTRODUCTION.....	4
2.1 Problem Statement.....	4
2.2 Proposed Solution	4
3. METHODOLOGY.....	5
3.1 Dataset & Preprocessing	5
3.2 Feature Engineering (The "Brain").....	5
3.3 Algorithm Selection: Random Forest	5
4. SYSTEM ARCHITECTURE	6
5. RESULTS AND ANALYSIS	7
5.1 Performance Metrics	7
5.2 Strengths.....	7
5.3 Limitations.....	7
6. USER INTERFACE (UI) IMPLEMENTATION	7
6.1 Main Dashboard & Input Mechanism	7
6.2 Detection Results (Safe vs. Malicious)	8
7. CONCLUSION & FUTURE WORK	9
7.1 Conclusion	9
7.2 Future Recommendations	9
References:.....	10

PHISHING URL DETECTOR

1. ABSTRACT

Phishing attacks remain one of the most critical cybersecurity threats, often bypassing traditional security measures by generating new, randomized domains (Zero-Day attacks). Traditional defense mechanisms, such as "Blacklisting," fail to detect these novel threats as they rely on static databases of known malicious sites.

This project proposes an **Intelligent Phishing URL Detector** that utilizes Machine Learning to identify malicious links based on their structural properties rather than a fixed database. Using a **Random Forest Classifier** trained on a dataset of over 500,000 URLs, the system extracts lexical features (such as URL length, digit density, and symbol usage) to classify URLs in real-time. The developed prototype achieved a testing accuracy of **85.68%**, demonstrating that structural analysis is a viable, proactive method for detecting unknown phishing threats.

2. INTRODUCTION

2.1 Problem Statement

The internet is flooded with thousands of new phishing websites daily. Traditional security tools (Antiviruses, Firewalls) rely heavily on **Signature-Based Detection** (Blacklists).

- **The Limitation:** A blacklist can only stop a URL it *already knows*.
- **The Threat:** Hackers use algorithms to generate random URLs (e.g., paypal-update-8823.com). Since these URLs are brand new, they bypass blacklists easily, putting users at risk of credential theft.

2.2 Proposed Solution

This project shifts the defense strategy from "Memorization" to "Pattern Recognition." We developed a Machine Learning model that analyzes the **Structure** (or "DNA") of a URL. By training the AI on the *behavior* of phishing URLs—such as excessive length or the use of deceptive subdomains—the system can flag malicious links even if they have never been reported before.

3. METHODOLOGY

3.1 Dataset & Preprocessing

We utilized a labeled dataset containing **500,000+ URLs**, balanced between "Legitimate" and "Phishing" samples.

- **Data Splitting:** The data was split into an **80% Training Set** (to teach the model) and a **20% Testing Set** (to evaluate performance on unseen data).

3.2 Feature Engineering (The "Brain")

The core of the project is **Feature Extraction**. The AI does not read the URL text like a human; it converts the URL into a mathematical vector of **8 structural features**:

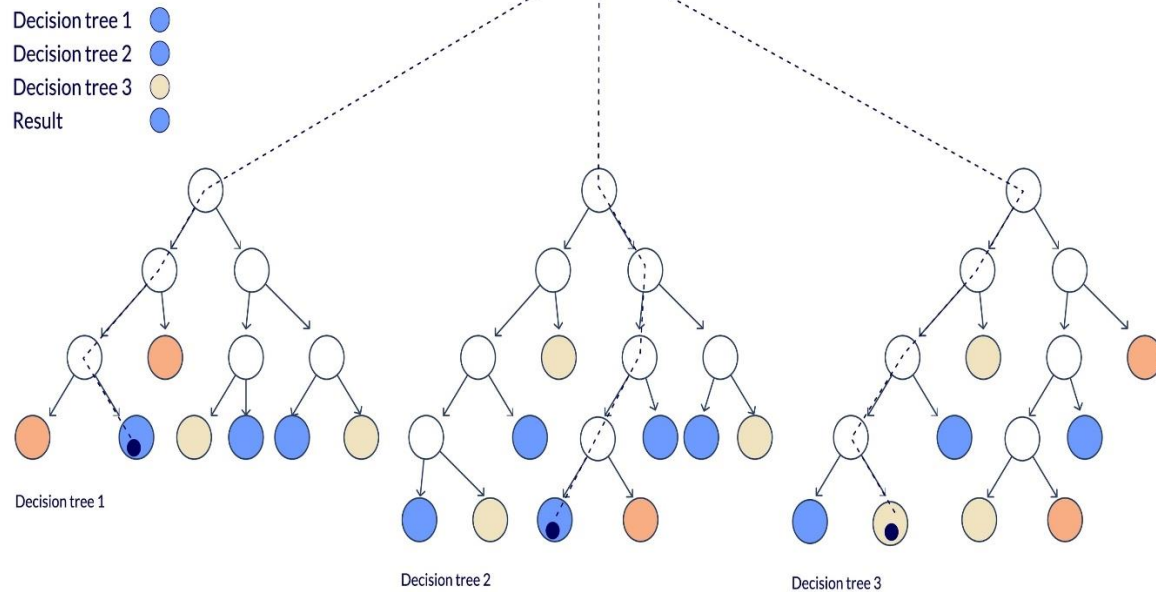
1. **URL Length:** Phishing URLs are often abnormally long to hide the actual domain on mobile screens.
2. **Dot Count:** Detecting subdomain abuse (e.g., `secure.login.update.bank.com`).
3. **@ Symbol:** Detecting redirection attacks (browsers ignore text before the @).
4. **Slash Count:** Detecting deep directory hiding.
5. **Hyphen Count:** Detecting typosquatting (e.g., `face-book.com` vs `facebook.com`).
6. **Digit Count:** Detecting raw IP addresses or obfuscation tokens.
7. **Sensitive Characters:** Counting query parameters (`?`, `=`, `&`) which are common in phishing kits.
8. **Digit Ratio:** The density of numbers relative to the total URL length.

3.3 Algorithm Selection: Random Forest

We selected the **Random Forest Classifier** over other algorithms (like SVM or Neural Networks) for three reasons:

1. **Ensemble Learning:** It builds 100 independent Decision Trees and takes a **Majority Vote**, reducing the risk of errors (overfitting).
2. **Tabular Suitability:** It performs exceptionally well on structured numerical data (our 8 features).
3. **Explainability:** It allows us to understand *why* a URL was blocked (e.g., "Blocked due to high anomaly score").

Random Forest



4. SYSTEM ARCHITECTURE

The system consists of two main components:

1. **The Training Pipeline (`train_model.py`):**
 - Loads the dataset.
 - Extracts features.
 - Trains the Random Forest agent.
 - Saves the trained "Brain" into a `.pkl` file.
 2. **The Detection Engine (`app.py`):**
 - A user-friendly web interface built with **Streamlit**.
 - Accepts a URL input from the user.
 - Extracts features in real-time.
 - Passes features to the loaded `.pkl` model.
 - Displays the result (Safe/Malicious) with a Confidence Score.
-

5. RESULTS AND ANALYSIS

5.1 Performance Metrics

Upon evaluating the model on the unseen Test Set (20% of data), the system achieved:

- **Accuracy: 85.68%**
- **Precision:** High reliability in flagging true threats.
- **Speed:** Classification occurs in milliseconds.

5.2 Strengths

- **Zero-Day Detection:** Successfully detects new random URLs not found in public blacklists.
- **Robustness:** The use of "Voting" (100 trees) makes the model resistant to minor noise.

5.3 Limitations

- **Shortened URLs:** Services like `bit.ly` hide the structure, sometimes confusing the model (requires unshortening logic).
 - **Perfect Mimicry:** If a hacker hacks a legitimate site (e.g., `university.edu/hacked`), the structure remains "Safe," and the model may miss it.
-

6. USER INTERFACE (UI) IMPLEMENTATION

The application interface was developed using **Streamlit**, a Python framework designed for rapid data science visualization. A custom CSS (Cascading Style Sheets) injection was applied to create a "Dark Mode" aesthetic, aligning with the cybersecurity context of the tool.

6.1 Main Dashboard & Input Mechanism

The landing page provides a clean, distraction-free environment for the user. It features a central input field designed to accept standard URL formats. The design minimizes cognitive load, allowing the user to focus solely on the analysis task.



Figure 7.1: The main interface showing the custom dark theme and URL input field.

6.2 Detection Results (Safe vs. Malicious)

Upon submitting a URL, the system provides immediate visual feedback using color-coded alerts:

- **Green Indicator:** Represents a "Safe" prediction. The system displays the confidence score (e.g., 98.5%) to reassure the user.

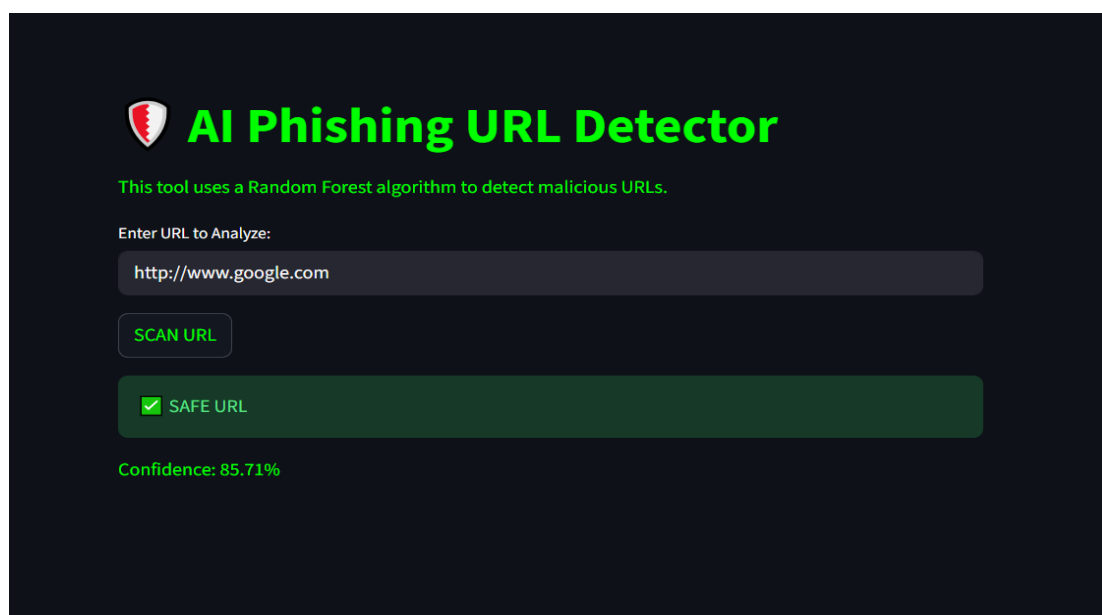


Figure 7.2: The system flagging a safe link with a 85.71% confidence score.

- **Red Indicator:** Represents a "Malicious" prediction. A warning alert is triggered, displaying the high probability of a threat and the reasoning (e.g., "High Anomaly Score").



Figure 7.3: The system flagging a phishing link with a 96.49% confidence score.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

This project successfully demonstrates that **Structural Analysis** is a powerful tool for cybersecurity. By automating the detection process using Artificial Intelligence, we can filter out the majority of phishing threats instantly, providing a first line of defense that is faster and more adaptive than traditional blacklists.

7.2 Future Recommendations

To transition this prototype into a production-ready security tool, I propose the following upgrades:

1. **Domain Age Analysis:** Integrating a **WHOIS API** to flag domains created less than 24 hours ago (a massive indicator of fraud).

2. **Content Analysis:** Scraping the HTML of the target page to check for keywords like "Login," "Password," or "Credit Card."
 3. **URL Unshortening:** Automatically expanding `bit.ly` links before analysis to reveal the true destination.
-

References:

- *Scikit-Learn Documentation*
- *Streamlit Framework for Data Apps*
- *Research Papers on Malicious URL Detection using Machine Learning*